

핸즈온 랩: Apache Spark로 데이터 분석하기

이 핸즈온 랩에서는 Fabric Lakehouse로 데이터를 수집하고, PySpark를 사용하여 데이터를 읽고 분석하는 방법을 배웁니다.

이 실습을 완료하는 데 약 **45분**이 소요됩니다.

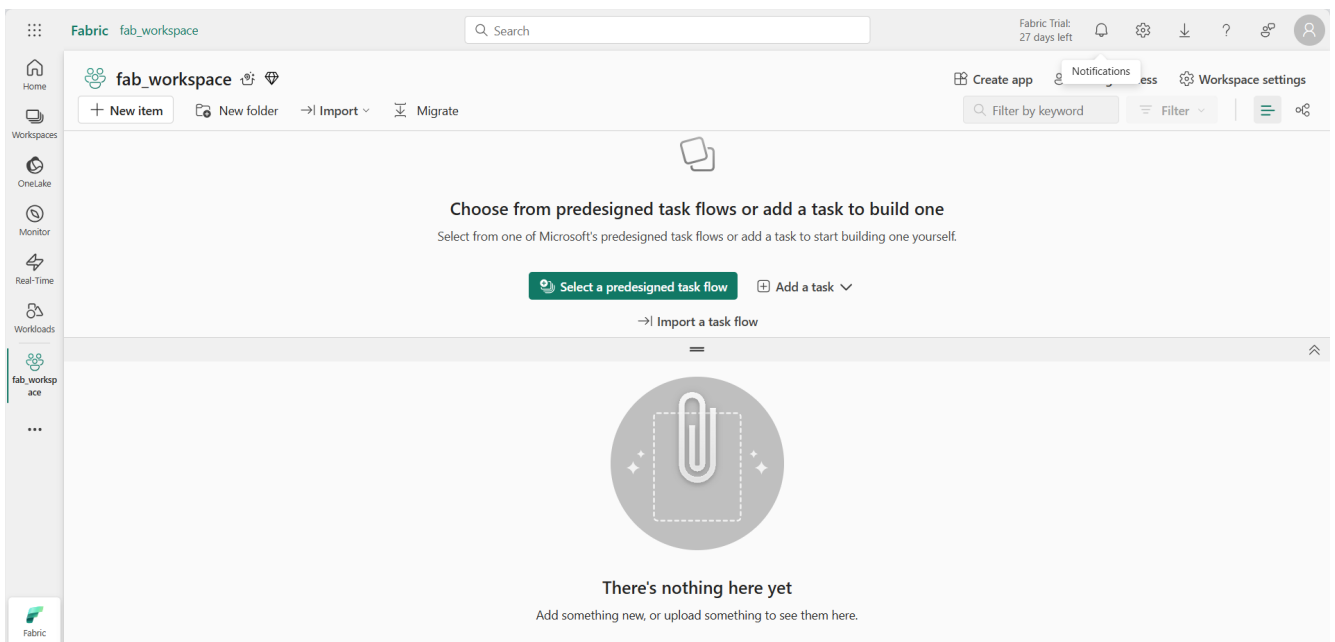
참고

이 실습을 완료하려면 [Microsoft Fabric 테넌트](#)에 대한 액세스 권한이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 용량이 활성화된 테넌트에 Workspace를 만들어야 합니다. 이 Workspace는 여러분의 프로젝트와 관련된 모든 Fabric 항목(Lakehouse, Notebook, 보고서 등)을 담는 컨테이너 역할을 합니다.

1. 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric-developer) `https://app.fabric.microsoft.com/home?experience=fabric-developer` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
3. 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial*, *Premium*, 또는 *Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
4. 새 Workspace가 열리면 처음에는 비어 있어야 합니다.



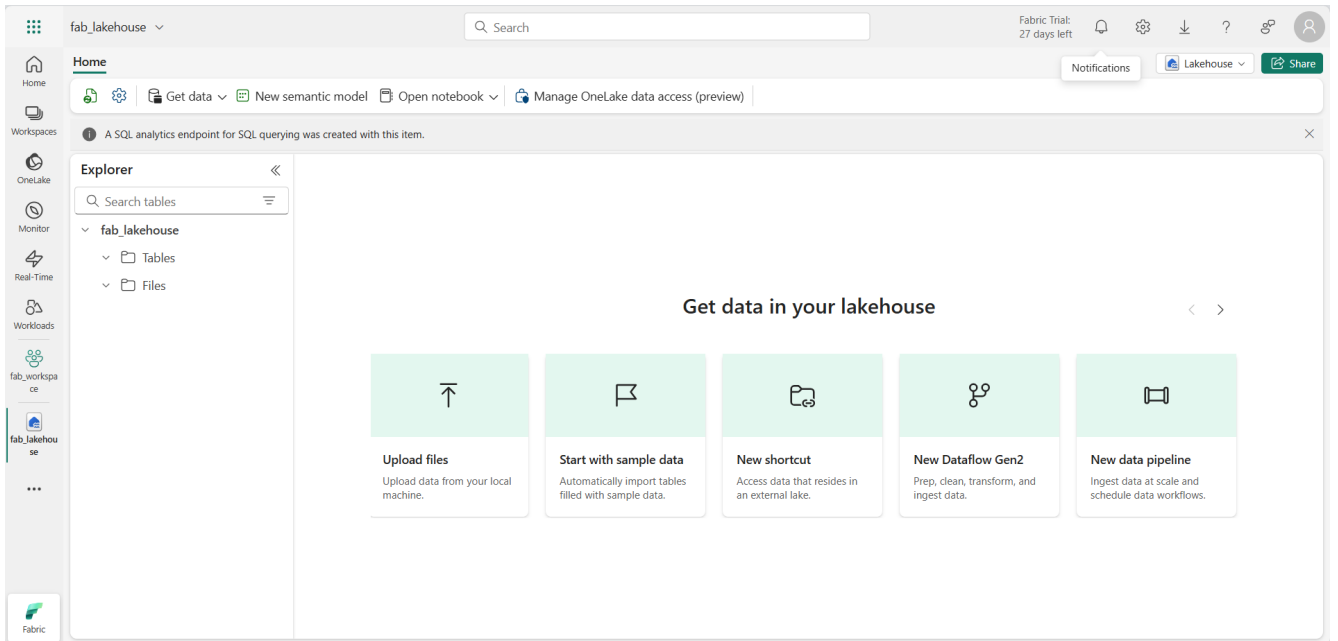
Lakehouse 만들기 및 파일 업로드

이제 Workspace가 준비되었으니 데이터를 위한 데이터 레이크하우스를 만들 차례입니다.

1. 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Engineering* 섹션에서 **Lakehouse**를 선택하고, 원하는 고유한 이름을 지정합니다.

참고: **Create** 옵션이 사이드바에 고정되어 있지 않다면, 먼저 줄임표(...) 옵션을 선택해야 합니다.

약 1분 후, 새로운 Lakehouse가 생성됩니다.



2. 새로운 Lakehouse를 살펴보면, 왼쪽의 **Lakehouse explorer** 창을 통해 Lakehouse의 테이블과 파일을 탐색할 수 있습니다.

이제 Lakehouse로 데이터를 수집할 수 있습니다. 여러 방법이 있지만, 여기서는 텍스트 파일이 담긴 폴더를 로컬 컴퓨터로 다운로드한 후 Lakehouse에 업로드하는 방식을 사용합니다.

핸즈온의 의미: 이 단계는 여러 개의 관련 파일을 한 번에 업로드하는 실제 시나리오를 경험하게 하여, Fabric Lakehouse가 개별 파일뿐만 아니라 폴더 구조도 쉽게 처리할 수 있음을 보여줍니다.

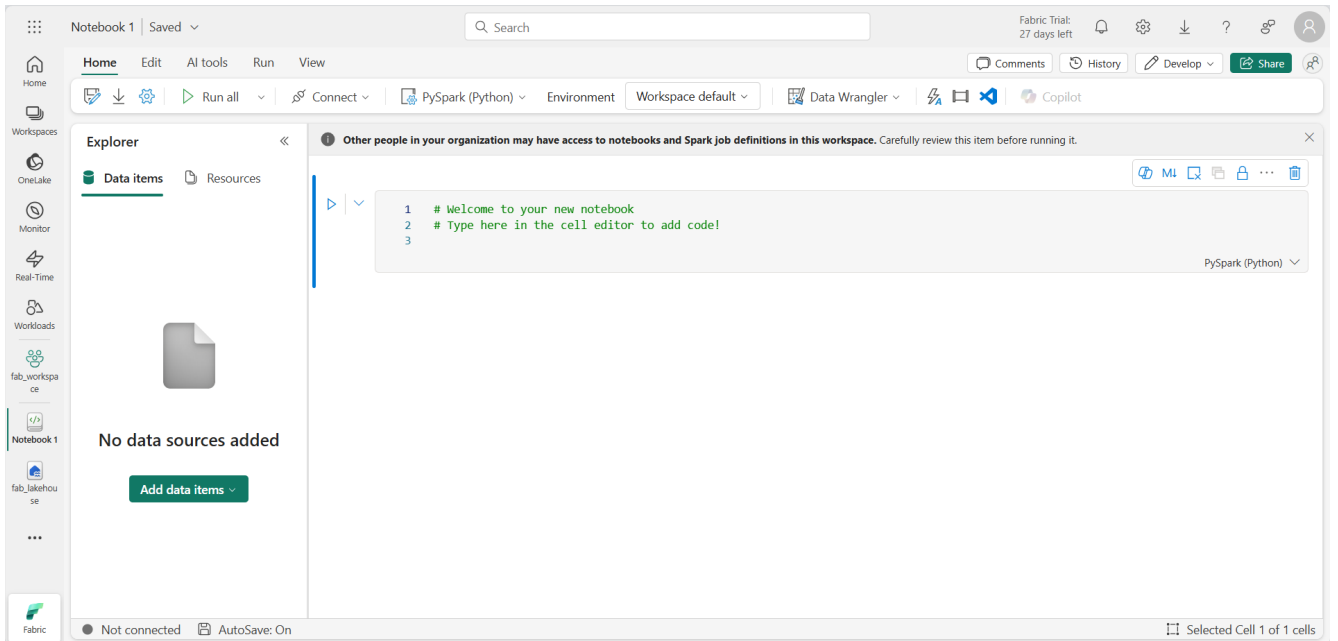
1. <https://github.com/MicrosoftLearning/dp-data/raw/main/orders.zip> 에서 데이터 파일을 다운로드합니다.
2. 압축된 아카이브 파일의 압축을 풀고, `2019.csv`, `2020.csv`, `2021.csv` 세 개의 CSV 파일이 포함된 `orders`라는 이름의 폴더가 있는지 확인합니다.
3. 새로운 Lakehouse로 돌아옵니다. **Explorer** 창에서 **Files** 폴더 옆에 있는 **...** 메뉴를 선택하고, **Upload**와 **Upload folder**를 차례로 선택합니다. 로컬 컴퓨터의 **orders** 폴더로 이동하여 **Upload**를 선택합니다.
4. 파일이 업로드된 후, **Files**를 확장하고 **orders** 폴더를 선택합니다. 아래와 같이 CSV 파일들이 업로드되었는지 확인합니다.

Notebook 만들기

이제 데이터를 다루기 위해 Fabric Notebook을 만들 수 있습니다.

개념 설명: Notebook은 코드를 작성하고 실행하며, 그 결과를 즉시 확인하고 문서를 함께 작성할 수 있는 대화형 환경입니다. 데이터 탐색, 분석, 시각화 작업에 매우 유용합니다.

1. 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Engineering* 섹션에서 **Notebook**을 선택합니다.
Notebook 1이라는 이름의 새 Notebook이 생성되고 열립니다.



2. Fabric은 생성하는 각 Notebook에 Notebook 1, Notebook 2 등과 같은 이름을 할당합니다. 메뉴의 **Home** 탭 위에 있는 이름 패널을 클릭하여 더 설명적인 이름으로 변경합니다.
3. 첫 번째 셀(현재 코드 셀)을 선택한 다음, 오른쪽 상단 도구 모음에서 **M↓** 버튼을 사용하여 Markdown 셀로 변환합니다. 그러면 셀에 포함된 텍스트가 서식이 지정된 텍스트로 표시됩니다. 다음과 같이 되도록 입력합니다.

![Screen picture of a Fabric notebook with a markdown cell.](Images/name-notebook-markdown.png)

편집이 끝나면 셀 외부의 Notebook 영역 아무 곳이나 클릭하여 편집을 마칩니다.

DataFrame 만들기

이제 Workspace, Lakehouse, Notebook을 모두 만들었으므로 데이터를 다룰 준비가 되었습니다. Fabric Notebook의 기본 언어이자 Spark에 최적화된 Python 버전인 PySpark를 사용할 것입니다.

참고

Fabric Notebook은 Scala, R, Spark SQL 등 여러 프로그래밍 언어를 지원합니다.

1. 왼쪽 막대에서 새 Workspace를 선택합니다. Lakehouse와 Notebook을 포함하여 Workspace에 포함된 항목 목록이 표시됩니다.
2. Lakehouse를 선택하여 **orders** 폴더를 포함한 Explorer 창을 표시합니다.
3. 상단 메뉴에서 **Open notebook, Existing notebook**을 차례로 선택한 다음, 이전에 만든 Notebook을 엽니다. 이제 Explorer 창 옆에 Notebook이 열려 있어야 합니다. Lakehouses를 확장하고, Files 목록을 확장한 후 orders 폴더를 선택합니다. 업로드한 CSV 파일이 아래와 같이 Notebook 편집기 옆에 나열됩니다.
4. **2019.csv**의 ... 메뉴에서 **Load data > Spark**를 선택합니다. 다음 코드가 새 코드 셀에 자동으로 생성됩니다.

```
df = spark.read.format("csv").option("header", "true").load("Files/orders/2019.csv")
# df now is a Spark DataFrame containing CSV data from "Files/orders/2019.csv".
display(df)
```

코드 설명:

- `spark.read.format("csv")`: Spark 세션을 사용하여 CSV 형식의 파일을 읽도록 지정합니다.
- `.option("header", "true")`: CSV 파일의 첫 번째 줄을 헤더(열 이름)로 사용하도록 설정합니다.
- `.load("Files/orders/2019.csv")`: 지정된 경로의 파일을 로드합니다.
- `display(df)`: 로드된 데이터를 포함하는 DataFrame `df`를 테이블 형식으로 보기 좋게 출력합니다.

팁

왼쪽의 Explorer 창은 « 아이콘을 사용하여 숨길 수 있습니다. 이렇게 하면 Notebook을 위한 공간이 더 넓어집니다.

5. 셀 왼쪽의 ▷ **Run cell**을 선택하여 코드를 실행합니다.

참고

Spark 코드를 처음 실행하면 Spark 세션이 시작됩니다. 이 과정은 몇 초 또는 그 이상 걸릴 수 있습니다. 동일한 세션 내에서 후속 실행은 더 빠릅니다.

6. 셀 코드 실행이 완료되면 셀 아래의 출력을 검토합니다. 다음과 같이 보여야 합니다.

7. 출력은 `2019.csv` 파일의 데이터를 열과 행으로 보여줍니다. 열 헤더에 데이터의 첫 번째 줄이 포함되어 있는 것을 확인하세요. 이를 수정하려면 코드의 첫 번째 줄을 다음과 같이 수정해야 합니다.

```
df = spark.read.format("csv").option("header", "false").load("Files/orders/2019.csv")
```

8. 코드를 다시 실행하여 DataFrame이 첫 번째 행을 데이터로 올바르게 식별하도록 합니다. 이제 열 이름이 `_c0`, `_c1` 등으로 변경된 것을 확인하세요.
9. 설명적인 열 이름은 데이터의 의미를 파악하는 데 도움이 됩니다. 의미 있는 열 이름을 만들려면 스키마와 데이터 타입을 정의해야 합니다. 또한 데이터 타입을 정의하기 위해 표준 Spark SQL 타입 세트를 가져와야 합니다. 기존 코드를 다음으로 교체하세요.

헤즈온의 의미: 이 단계는 원시 데이터에 명확한 구조(스키마)와 타입(자료형)을 부여하는 과정입니다. 데이터의 정확성과 처리 성능을 보장하기 위해 명시적으로 스키마를 정의하는 방법을 직접 경험해 봅니다.

```
from pyspark.sql.types import *

orderSchema = StructType([
    StructField("SalesOrderNumber", StringType()),
    StructField("SalesOrderLineNumber", IntegerType()),
    StructField("OrderDate", DateType()),
    StructField("CustomerName", StringType()),
    StructField("Email", StringType()),
    StructField("Item", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", FloatType()),
    StructField("Tax", FloatType())
])
```

```

    })

    df = spark.read.format("csv").schema(orderSchema).load("Files/orders/2019.csv")

    display(df)

```

10. 셀을 실행하고 출력을 검토합니다.(StructType은 pyspark.sql.types 모듈에 정의된 클래스(Class))

11. 이 DataFrame은 2019.csv 파일의 데이터만 포함합니다. orders 폴더의 모든 파일을 읽도록 파일 경로에 * 와일드카드를 사용하도록 코드를 수정합니다.

코드 설명: Files/orders/*.csv 에서 * 는 "모든 문자열"을 의미하는 와일드카드로, orders 폴더 내에 .csv 로 끝나는 모든 파일을 읽어 하나의 DataFrame으로 합치도록 Spark에 지시합니다.

```

from pyspark.sql.types import *

orderSchema = StructType([
    StructField("SalesOrderNumber", StringType()),
    StructField("SalesOrderLineNumber", IntegerType()),
    StructField("OrderDate", DateType()),
    StructField("CustomerName", StringType()),
    StructField("Email", StringType()),
    StructField("Item", StringType()),
    StructField("Quantity", IntegerType()),
    StructField("UnitPrice", FloatType()),
    StructField("Tax", FloatType())
])

df = spark.read.format("csv").schema(orderSchema).load("Files/orders/*.csv")

display(df)

```

12. 수정된 코드를 실행하면 2019년, 2020년, 2021년의 판매 데이터를 볼 수 있습니다. 행의 일부만 표시되므로 모든 연도의 행이 보이지 않을 수 있습니다.

참고

결과 옆에 있는 ...를 선택하여 셀의 출력을 숨기거나 표시할 수 있습니다. 이렇게 하면 Notebook에서 작업하기가 더 쉬워집니다.

DataFrame에서 데이터 탐색하기

DataFrame 객체는 데이터를 필터링, 그룹화 및 조작하는 기능과 같은 추가 기능을 제공합니다.

DataFrame 필터링하기

1. 현재 셀 또는 그 출력 위나 아래로 마우스를 가져가면 나타나는 + **Code**를 선택하여 코드 셀을 추가합니다. 또는 리본 메뉴에서 **Edit**을 선택하고 + **Add code cell below**를 선택합니다.
2. 다음 코드는 데이터를 필터링하여 두 개의 열만 반환합니다. 또한 count 와 distinct 를 사용하여 레코드 수를 요약합니다.

```
customers = df['CustomerName', 'Email']

print(customers.count())
print(customers.distinct().count())

display(customers.distinct())
```

코드 설명:

- `customers = df['CustomerName', 'Email']`: 기존 `df` DataFrame에서 `CustomerName` 과 `Email` 두 열만 선택하여 새로운 `customers` DataFrame을 생성합니다.
- `customers.count()`: `customers` DataFrame의 전체 행 수를 계산합니다.
- `customers.distinct().count()`: 중복된 행을 제거한 후의 고유한 행 수를 계산합니다.

3. 코드를 실행하고 출력을 검사합니다.

- 코드는 원래 `df` DataFrame의 열 일부를 포함하는 **customers**라는 새 DataFrame을 만듭니다. DataFrame 변환을 수행할 때 원본 DataFrame을 수정하지 않고 새 DataFrame을 반환합니다.
- 동일한 결과를 얻는 또 다른 방법은 `select` 메서드를 사용하는 것입니다:

```
customers = df.select("CustomerName", "Email")
```

4. `select` 와 `where` 함수를 사용하여 코드의 첫 번째 줄을 다음과 같이 수정합니다.

```
customers = df.select("CustomerName", "Email").where(df['Item']=='Road-250 Red, 52')
print(customers.count())
print(customers.distinct().count())

display(customers.distinct())
```

코드 설명: `where(df['Item']=='Road-250 Red, 52')` 는 `Item` 열의 값이 'Road-250 Red, 52'인 행만 필터링하는 조건입니다. 여러 함수를 "연결(chaining)"하여 한 함수의 출력이 다음 함수의 입력이 되도록 할 수 있습니다.

5. 수정된 코드를 실행하여 'Road-250 Red, 52' 제품을 구매한 고객만 선택합니다.

6. Item 컬럼도 추가해봅니다.

```
customers = df.select('CustomerName', 'Email', 'Item').where(df['Item']=='Road-250 Red, 52')
```

DataFrame에서 데이터 집계 및 그룹화하기

1. 코드 셀을 추가하고 다음 코드를 입력합니다.

```
productSales = df.select("Item", "Quantity").groupBy("Item").sum()

display(productSales)
```

코드 설명:

- `.groupBy("Item"): Item` 열을 기준으로 데이터를 그룹화합니다.
- `.sum()`: 그룹화된 각 `Item`에 대해 나머지 숫자 열(여기서는 `Quantity`)의 합계를 계산합니다.

2. 코드를 실행합니다. 결과는 제품별로 그룹화된 주문 수량의 합계를 보여줍니다.

3. Notebook에 다른 코드 셀을 추가하고 다음 코드를 입력합니다.

```
from pyspark.sql.functions import *

yearlySales =
df.select(year(col("OrderDate")).alias("Year")).groupBy("Year").count().orderBy("Year")

display(yearlySales)
```

코드 설명:

- `from pyspark.sql.functions import *`: `year`와 같은 Spark SQL 함수를 사용하기 위해 라이브러리를 가져옵니다.
- `select(year(col("OrderDate")).alias("Year")): OrderDate` 열에서 연도만 추출하고, 이 새로운 열의 이름을 `Year`로 지정합니다.
- `.groupBy("Year").count()`: `Year`별로 그룹화하고 각 그룹의 행 수를 셉니다.
- `.orderBy("Year")`: 결과를 연도순으로 정렬합니다.

4. 셀을 실행합니다. 출력을 검사합니다. 결과는 이제 연도별 판매 주문 수를 보여줍니다.

Spark를 사용하여 데이터 파일 변환하기

데이터 엔지니어와 데이터 과학자의 일반적인 작업 중 하나는 추가적인 다운스트림 처리나 분석을 위해 데이터를 변환하는 것입니다.

DataFrame 메서드와 함수를 사용하여 데이터 변환하기

1. Notebook에 코드 셀을 추가하고 다음을 입력합니다.

```

from pyspark.sql.functions import *

# Create Year and Month columns
transformed_df = df.withColumn("Year", year(col("OrderDate"))).withColumn("Month",
month(col("OrderDate")))

# Create the new FirstName and LastName fields
transformed_df = transformed_df.withColumn("FirstName", split(col("CustomerName"), "
").getItem(0)).withColumn("LastName", split(col("CustomerName"), " ").getItem(1))

# Filter and reorder columns
transformed_df = transformed_df["SalesOrderNumber", "SalesOrderLineNumber",
"OrderDate", "Year", "Month", "FirstName", "LastName", "Email", "Item", "Quantity",
"UnitPrice", "Tax"]

# Display the first five orders
display(transformed_df.limit(5))

```

코드 설명:

- `.withColumn("Year", ...)`: `OrderDate` 열을 기반으로 `Year` 와 `Month` 라는 새 열을 추가합니다.
 - `.withColumn("FirstName", split(...).getItem(0))`: `CustomerName` 을 공백(" ")으로 분리하고 첫 번째(`getItem(0)`) 요소를 `FirstName` 으로, 두 번째(`getItem(1)`) 요소를 `LastName` 으로 하는 새 열을 만듭니다.
 - `transformed_df[...]`: 필요한 열만 선택하고 순서를 재정렬하여 최종 `DataFrame`을 만듭니다. `CustomerName` 열은 제거됩니다.
2. 셀을 실행합니다. 원본 주문 데이터에서 다음과 같은 변환이 적용된 새 `DataFrame`이 생성됩니다.
 3. 출력을 검토하고 데이터에 변환이 적용되었는지 확인합니다.

팁

`DataFrame` 객체에 대해 더 자세히 알아보려면 [Apache Spark dataframe](#) 문서를 참조하세요.

변환된 데이터 저장하기

이 시점에서 변환된 데이터를 저장하여 추가 분석에 사용할 수 있습니다.

개념 설명: **Parquet**는 데이터를 효율적으로 저장하고 대부분의 대규모 데이터 분석 시스템에서 지원되기 때문에 널리 사용되는 열(columnar) 기반 데이터 저장 형식입니다. CSV와 같은 형식을 Parquet로 변환하는 것은 데이터 엔지니어링의 일반적인 작업입니다.

1. 변환된 `DataFrame`을 Parquet 형식으로 저장하려면 코드 셀을 추가하고 다음 코드를 추가합니다.

```

transformed_df.write.mode("overwrite").parquet('Files/transformed_data/orders')

print ("Transformed data saved!")

```

코드 설명:

- `.write`: `DataFrame`을 저장하기 위한 메서드입니다.

- `.mode("overwrite")` : 동일한 이름의 파일이나 폴더가 이미 존재하면 덮어씁니다.
 - `.parquet(...)` : 데이터를 Parquet 형식으로 지정된 경로에 저장합니다.
2. 셀을 실행하고 데이터가 저장되었다는 메시지를 기다립니다. 그런 다음 왼쪽의 **Explorer** 창에서 Files 노드의 ... 메뉴에서 **Refresh**를 선택합니다. `transformed_data` 폴더를 선택하여 그 안에 하나 이상의 Parquet 파일을 포함하는 `orders` 라는 새 폴더가 포함되어 있는지 확인합니다.
 3. 다음 코드로 셀을 추가합니다.

```
orders_df = spark.read.format("parquet").load("Files/transformed_data/orders")
display(orders_df)
```

4. 셀을 실행합니다. `transformed_data/orders` 폴더의 Parquet 파일에서 새 DataFrame이 생성됩니다. 결과가 Parquet 파일에서 로드된 주문 데이터를 보여주는지 확인합니다.

파티션된 파일에 데이터 저장하기

개념 설명: Partitioning(파티셔닝)은 대용량 데이터를 다룰 때 성능을 크게 향상시키고 데이터 필터링을 쉽게 할 수 있는 기술입니다. 데이터를 특정 열(예: 연도, 월)의 값에 따라 하위 폴더로 나누어 저장합니다. 이렇게 하면 해당 열을 기준으로 필터링하는 쿼리(예: `WHERE Year = 2021`)가 전체 데이터를 스캔할 필요 없이 해당 폴더만 읽게 되어 속도가 매우 빨라집니다.

1. Year와 Month로 데이터를 파티셔닝하여 데이터 프레임을 저장하는 코드로 셀을 추가합니다.

```
orders_df.write.partitionBy("Year", "Month").mode("overwrite").parquet("Files/partitioned_data")

print("Transformed data saved!")
```

코드 설명: `.partitionBy("Year", "Month")` 는 DataFrame을 저장할 때 `Year` 와 `Month` 열의 값을 기준으로 폴더 계층 구조를 만들어 데이터를 분할하도록 지시합니다.

2. 셀을 실행하고 데이터가 저장되었다는 메시지를 기다립니다. 그런 다음 왼쪽의 **Lakehouses** 창에서 Files 노드의 ... 메뉴에서 **Refresh**를 선택하고 `partitioned_data` 폴더를 확장하여 `Year=xxxx` 라는 이름의 폴더 계층 구조가 포함되어 있는지 확인합니다. 각 폴더에는 `Month=xxxx` 라는 이름의 폴더가 포함됩니다. 각 월 폴더에는 해당 월의 주문이 담긴 Parquet 파일이 있습니다.
3. `orders.parquet` 파일에서 새 DataFrame을 로드하는 다음 코드로 새 셀을 추가합니다.

```
orders_2021_df =
spark.read.format("parquet").load("Files/partitioned_data/Year=2021/Month=*")

display(orders_2021_df)
```

4. 셀을 실행하고 결과가 2021년 판매 주문 데이터를 보여주는지 확인합니다. 경로에 지정된 파티셔닝 열(Year 및 Month)은 DataFrame에 포함되지 않습니다.

테이블 및 SQL 작업하기

지금까지 DataFrame 객체의 네이티브 메서드를 사용하여 파일에서 데이터를 쿼리하고 분석하는 방법을 보았습니다. 하지만 SQL 구문을 사용하여 테이블로 작업하는 것이 더 편할 수 있습니다. Spark는 관계형 테이블을 정의할 수 있는 메타스토어를 제공합니다.

개념 설명: Spark SQL 라이브러리는 **Metastore**에 있는 테이블을 쿼리하기 위해 SQL 문을 사용하는 것을 지원합니다. 이는 데이터 레이크의 유연성과 관계형 데이터 웨어하우스의 구조화된 데이터 스키마 및 SQL 기반 쿼리를 결합하여 **"Data Lakehouse"**라는 용어를 탄생시켰습니다.

테이블 생성하기

Spark 메타스토어의 테이블은 데이터 레이크의 파일에 대한 관계형 추상화입니다. 테이블은 메타스토어에 의해 *managed*되거나, 메타스토어와 독립적으로 관리되는 *external*일 수 있습니다.

1. Notebook에 코드 셀을 추가하고 판매 주문 데이터의 DataFrame을 `salesorders` 라는 이름의 테이블로 저장하는 다음 코드를 입력합니다.

```
# Create a new table
df.write.format("delta").saveAsTable("salesorders")

# Get the table description
spark.sql("DESCRIBE EXTENDED salesorders").show(truncate=False)
```

참고

이 예에서는 명시적인 경로가 제공되지 않으므로 테이블 파일은 메타스토어에 의해 관리됩니다. 또한 테이블은 **Delta** 형식으로 저장되어 테이블에 관계형 데이터베이스 기능을 추가합니다. 여기에는 트랜잭션 지원, 행 버전 관리 및 기타 유용한 기능이 포함됩니다. Fabric의 데이터 레이크하우스에서는 Delta 형식으로 테이블을 만드는 것이 선호됩니다.

2. 코드 셀을 실행하고 새 테이블의 정의를 설명하는 출력을 검토합니다.
3. **Explorer** 창의 Tables 폴더에 있는 ... 메뉴에서 **Refresh**를 선택합니다. 그런 다음 **Tables** 노드를 확장하고 **salesorders** 테이블이 생성되었는지 확인합니다.
4. `salesorders` 테이블의 ... 메뉴에서 **Load data > Spark**를 선택합니다. 다음과 유사한 코드가 포함된 새 코드 셀이 추가됩니다.

```
df = spark.sql("SELECT * FROM [your_lakehouse].salesorders LIMIT 1000")

display(df)
```

5. 새 코드를 실행합니다. 이 코드는 Spark SQL 라이브러리를 사용하여 `salesorder` 테이블에 대한 SQL 쿼리를 PySpark 코드에 포함시키고 쿼리 결과를 DataFrame으로 로드합니다.

셀에서 SQL 코드 실행하기

PySpark 코드가 포함된 셀에 SQL 문을 포함할 수 있는 것도 유용하지만, 데이터 분석가들은 종종 SQL로 직접 작업하기를 원합니다.

핸즈온의 의미: `%%sql` 매직 커맨드를 사용하여 Notebook 셀의 언어를 PySpark에서 Spark SQL로 전환하는 방법을 배웁니다. 이를 통해 SQL에 익숙한 사용자들이 복잡한 Python 코드 없이도 친숙한 SQL 구문으로 직접 데이터를 분석할 수 있음을 체험합니다.

1. Notebook에 새 코드 셀을 추가하고 다음 코드를 입력합니다.

```
%%sql
SELECT YEAR(OrderDate) AS OrderYear,
       SUM((UnitPrice * Quantity) + Tax) AS GrossRevenue
FROM salesorders
GROUP BY YEAR(OrderDate)
ORDER BY OrderYear;
```

2. 셀을 실행하고 결과를 검토합니다. 다음을 확인하세요.

- 셀 시작 부분의 `%%sql` 명령어(magic이라고 함)는 언어를 PySpark 대신 Spark SQL로 변경합니다.
- SQL 코드는 이전에 만든 `salesorders` 테이블을 참조합니다.
- SQL 쿼리의 출력이 셀 아래의 결과로 자동으로 표시됩니다.

참고

Spark SQL 및 데이터프레임에 대한 자세한 내용은 [Apache Spark SQL](#) 문서를 참조하세요.

Spark로 데이터 시각화하기

차트는 수천 개의 데이터 행을 스캔하는 것보다 패턴과 추세를 더 빨리 파악하는 데 도움이 됩니다. Fabric Notebook에는 내장된 차트 보기가 있지만 복잡한 차트를 위해 설계된 것은 아닙니다. DataFrame의 데이터에서 차트를 만드는 방법을 더 잘 제어하려면 *matplotlib* 또는 *seaborn*과 같은 Python 그래픽 라이브러리를 사용하세요.

결과를 차트로 보기

1. 새 코드 셀을 추가하고 다음 코드를 입력합니다.

```
%%sql
SELECT * FROM salesorders
```

2. 코드를 실행하여 이전에 만든 `salesorders` 뷰의 데이터를 표시합니다. 셀 아래의 결과 섹션에서 **+ New chart**를 선택합니다.
3. 결과 섹션의 오른쪽 하단에 있는 **Build my own** 버튼을 사용하고 차트 설정을 지정합니다.
 - Chart type: `Bar chart`
 - X-axis: `Item`
 - Y-axis: `Quantity`
 - Series Group: 비워 둠
 - Aggregation: `Sum`
 - Missing and NULL values: `Display as 0`
 - Stacked: 선택 해제

4. 차트는 다음과 같이 보여야 합니다.

matplotlib 시작하기

개념 설명: **matplotlib**는 Python의 핵심 플로팅 라이브러리로, 다른 많은 라이브러리의 기반이 되며 차트 생성에 있어 뛰어난 유연성을 제공합니다.

1. 새 코드 셀을 추가하고 다음 코드를 입력합니다.

```
sqlQuery = "SELECT CAST(YEAR(OrderDate) AS CHAR(4)) AS OrderYear, \
            SUM((UnitPrice * Quantity) + Tax) AS GrossRevenue, \
            COUNT(DISTINCT SalesOrderNumber) AS YearlyCounts \
            FROM salesorders \
            GROUP BY CAST(YEAR(OrderDate) AS CHAR(4)) \
            ORDER BY OrderYear"
df_spark = spark.sql(sqlQuery)
df_spark.show()
```

2. 코드를 실행합니다. 연간 수익과 주문 수를 포함하는 Spark DataFrame을 반환합니다.

3. 새 코드 셀을 추가하고 다음 코드를 추가합니다.

```
from matplotlib import pyplot as plt

# matplotlib requires a Pandas dataframe, not a Spark one
df_sales = df_spark.toPandas()

# Create a bar plot of revenue by year
plt.bar(x=df_sales['OrderYear'], height=df_sales['GrossRevenue'])

# Display the plot
plt.show()
```

코드 설명:

- `df_spark.toPandas()`: `matplotlib`은 Pandas DataFrame과 함께 작동하므로, Spark DataFrame을 Pandas DataFrame으로 변환하는 중요한 단계입니다.
- `plt.bar(...)`: x축과 y축(높이)을 지정하여 막대그래프를 생성합니다.
- `plt.show()`: 생성된 플롯을 화면에 표시합니다.

4. 셀을 실행하고 연도별 총 총수익을 나타내는 막대 차트로 구성된 결과를 검토합니다.

5. 차트를 플로팅하기 위해 코드를 다음과 같이 수정합니다.

```
from matplotlib import pyplot as plt

# Clear the plot area
plt.clf()

# Create a bar plot of revenue by year
```

```
plt.bar(x=df_sales['OrderYear'], height=df_sales['GrossRevenue'], color='orange')

# Customize the chart
plt.title('Revenue by Year')
plt.xlabel('Year')
plt.ylabel('Revenue')
plt.grid(color='#95a5a6', linestyle='--', linewidth=2, axis='y', alpha=0.7)
plt.xticks(rotation=45)

# Show the figure
plt.show()
```

6. 코드 셀을 다시 실행하고 결과를 봅니다. 이제 차트가 더 이해하기 쉬워졌습니다.

7. 플롯은 Figure 내에 포함됩니다. 이전 예에서는 암시적으로 생성되었지만 명시적으로 생성할 수도 있습니다.

```
from matplotlib import pyplot as plt

# Clear the plot area
plt.clf()

# Create a Figure
fig = plt.figure(figsize=(8,3))
# ... (나머지 코드는 동일)
```

8. Figure는 여러 개의 서브플롯을 포함할 수 있으며, 각 서브플롯은 자체 축에 있습니다. 코드를 다음과 같이 수정하여 차트를 플로팅합니다.

```
from matplotlib import pyplot as plt

# Clear the plot area
plt.clf()

# Create a figure for 2 subplots (1 row, 2 columns)
fig, ax = plt.subplots(1, 2, figsize = (10,4))

# Create a bar plot of revenue by year on the first axis
ax[0].bar(x=df_sales['OrderYear'], height=df_sales['GrossRevenue'], color='orange')
ax[0].set_title('Revenue by Year')

# Create a pie chart of yearly order counts on the second axis
ax[1].pie(df_sales['YearlyCounts'])
ax[1].set_title('Orders per Year')
ax[1].legend(df_sales['OrderYear'])

# Add a title to the Figure
fig.suptitle('Sales Data')

# Show the figure
plt.show()
```

9. 코드 셀을 다시 실행하고 결과를 봅니다.

참고

matplotlib로 플로팅하는 방법에 대해 더 자세히 알아보려면 [matplotlib](#) 문서를 참조하세요.

seaborn 라이브러리 사용하기

개념 설명: **seaborn**은 `matplotlib`을 기반으로 구축된 라이브러리로, 복잡성을 추상화하고 기능을 향상시켜 통계적 차트를 더 쉽고 아름답게 만들 수 있도록 도와줍니다.

1. Notebook에 새 코드 셀을 추가하고 다음 코드를 입력합니다.

```
import seaborn as sns

# Clear the plot area
plt.clf()

# Create a bar chart
ax = sns.barplot(x="OrderYear", y="GrossRevenue", data=df_sales)

plt.show()
```

2. 코드를 실행하여 seaborn 라이브러리를 사용하여 생성된 막대 차트를 표시합니다.

3. 코드를 다음과 같이 수정합니다.

```
import seaborn as sns

# Clear the plot area
plt.clf()

# Set the visual theme for seaborn
sns.set_theme(style="whitegrid")

# Create a bar chart
ax = sns.barplot(x="OrderYear", y="GrossRevenue", data=df_sales)

plt.show()
```

4. 수정된 코드를 실행하고 seaborn이 플롯에 대한 색상 테마를 설정할 수 있음을 확인합니다.

5. 코드를 다음과 같이 다시 수정합니다.

```
import seaborn as sns

# Clear the plot area
plt.clf()

# Create a line chart
ax = sns.lineplot(x="OrderYear", y="GrossRevenue", data=df_sales)

plt.show()
```

6. 수정된 코드를 실행하여 연간 수익을 꺾은선형 차트로 봅니다.

참고

seaborn으로 플로팅하는 방법에 대해 더 자세히 알아보려면 [seaborn](#) 문서를 참조하세요.

리소스 정리

이 실습에서는 Spark를 사용하여 Microsoft Fabric에서 데이터를 사용하는 방법을 배웠습니다.

데이터 탐색을 마쳤으면 Spark 세션을 종료하고 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

1. Notebook 메뉴에서 **Stop session**을 선택하여 Spark 세션을 종료합니다.
2. 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
3. **Workspace settings**를 선택하고 **General** 섹션에서 아래로 스크롤하여 **Remove this workspace**를 선택합니다.
4. **Delete**를 선택하여 Workspace를 삭제합니다.